

ArrayList.c Explained

🔖 개요

`ArrayList.c`는 배열을 기본 구조로 가장 편리한 프로그램적 데이터 구조 중 하나인 **배열 리스트(Array List)** 구현 예제입니다.

▶ 실행 순서

1. `init()`으로 구성한 리스트 구성
2. 내림 추가 (`insert_last`, `insert`)
3. 값 검색 (`get_entry`)
4. 추가된 데이터 표시 (`print_list`)
5. 삭제 (`delete`)

확장적인 규정을 가지고 있음:

```
#define max_list_size 100
```

- 목록 크기가 100으로 한정됩니다.

해당 함수 설명

1. `init(arraylisttype *l)`

- 배열 리스트 초기화 (사이즈 0)

2. `is_empty(arraylisttype *l)`

- 빠지어 있는지 판단 (`size == 0`)

3. `is_full(arraylisttype *l)`

- 꼭 채어있는지 판단 (`size == max`)

4. `get_entry(arraylisttype *l, int pos)`

- 각 위치의 값 검색
- 위치 오류시 `error()` 함수 호출

5. `print_list(arraylisttype *l)`

- 리스트 목록 표시 (`a -> b -> ...`)

6. `insert_last(arraylisttype *l, element item)`

- 바로 마지막에 값 추가

7. `insert(arraylisttype *l, int pos, element item)`

- 중간에 값 추가 (pos를 기준으로 값 이동 등 처리)

8. `delete(arraylisttype *l, int pos)` (파일 뒷부분에 구현될 가능성 있음)

- pos 위치의 값 삭제, 이후 값 다시 정렬

주요 개념

- 정수 배열을 이용
- 최대 크기 한정된 건지 검색 필요
- 업데이트: 추가/삭제 시 값 이동이 사용되며, 각 element 의 이동이 필요

예제 사용

```
arraylisttype list;
init(&list);
insert_last(&list, 10);
insert(&list, 1, 20);
delete(&list, 0);
print_list(&list); // 값이 반영됨
```

이 파일을 유용하게 하는 점

- 목록 전체에 대한 변환(추가/삭제)가 복잡겠지만, 그 결과를 직관적으로 표시하고, 오류 처리도 각해되어 있음.